

Validacion Experimental de H1

De la intuicion a la hipotesis. De la hipotesis al experimento. Del experimento a la evidencia.

HIPOTESIS CENTRAL

En discos con presupuesto de lectura limitado por degradacion fisica, una estrategia de adquisicion basada en metadatos recuperables incrementa la cantidad de archivos recuperados y/o reduce el numero de lecturas necesarias respecto a una estrategia secuencial.

Capitulo 1. De la intuicion a la hipotesis	3
1.1 Por que un laboratorio, no un motor	3
1.2 La evolucion de la pregunta	3
Capitulo 2. H1: Hipotesis Central	4
2.1 Descomposicion de H1	4
2.2 Lo que H1 NO afirma	5
2.3 Variables de H1	5
Capitulo 3. El simulador: resultados y limites	6
3.1 Resultados del simulador	6
3.2 El error detectado y corregido	6
3.3 Limitaciones del simulador	7
Capitulo 4. El siguiente experimento: imagenes reales	7
4.1 Diseno del experimento	7
4.2 Patrones de corrupcion	8
4.3 Presupuestos de lectura	9
Capitulo 5. Metricas ampliadas	9
5.1 Metricas primarias	9
5.2 Metricas de eficiencia	10
5.3 Metricas de estres	10
5.4 Escenario hipotetico: la ventaja oculta	11
Capitulo 6. Cuando el MFT falla	11
6.1 Arbol de decisiones de diagnostico	11
6.2 Fuentes de informacion alternativas	12
6.3 El experimento de degradacion del MFT	13
Capitulo 7. Matriz de ataque a H1	13
7.1 Escenarios de ataque	13
7.2 Priorizacion de ataques	14
7.3 Criterios de resultado por ataque	14
Capitulo 8. Criterios de decision	15

8.1 Umbrales de decision 15

8.2 Escenarios de decision 16

8.3 La metrica de eficiencia como salvavidas 16

8.4 Roadmap post-experimento 17

Capitulo 1. De la intuicion a la hipotesis

Este documento no describe un producto. Describe un experimento. La diferencia es fundamental: un producto se construye sobre convicciones; un experimento se construye sobre dudas. Hemos llegado a un punto donde la intuicion inicial —que leer primero los metadatos del sistema de archivos deberia mejorar la recuperacion de datos en discos danados— ha madurado lo suficiente como para convertirse en una hipotesis tecnica formal. Pero no hemos demostrado que esa intuicion sea correcta. Y eso, antes de escribir miles de lineas de codigo, es lo que debemos resolver.

El recorrido hasta aqui ha sido deliberadamente conservador. En la Fase 1 analizamos el mercado y los competidores. En la Fase 2 intentamos refutar la idea con preguntas agresivas. En la Fase 3 disenamos la ingenieria del motor de recuperacion. Y en cada paso, la idea sobrevivio — pero sobrevivir al escrutinio teorico no es lo mismo que sobrevivir a la evidencia experimental. Hay una diferencia abismal entre "tiene sentido que funcione" y "medimos que funciona". Este documento es el puente entre ambas.

1.1 Por que un laboratorio, no un motor

La tentacion natural de cualquier ingeniero es programar. Tenemos la arquitectura, tenemos los componentes identificados, tenemos el roadmap. Podriamos abrir el IDE y empezar a construir. Pero eso seria cometer el error mas costoso de todos: construir un producto sobre una hipotesis no verificada. Si la priorizacion MFT-first resulta no ofrecer ventaja significativa en condiciones reales, todo el motor de decisiones, todo el sistema de diagnostico, toda la logica de adquisicion priorizada seria codigo inutil. No es que no funcionaria — es que resolveria el problema equivocado.

El Benchmark Lab es, en esencia, un mecanismo de proteccion. No protege contra fallos tecnicos del codigo; protege contra fallos fundamentales del concepto. Si la hipotesis es correcta, el laboratorio nos dara la confianza para invertir meses de desarrollo. Si es incorrecta, nos habra ahorrado meses de desarrollo en la direccion equivocada. Y si es parcialmente correcta —si funciona solo en ciertas condiciones—, nos dira exactamente cuales son esas condiciones, permitiendonos enfocar el producto con precision quirurgica.

1.2 La evolucion de la pregunta

La pregunta original era amplia y vaga: "Podemos construir un motor de recuperacion mejor que los existentes?" Con cada fase, la pregunta se fue refinando. La Fase 1 la redujo a "Hay una oportunidad tecnica real?" La Fase 2 la estrecho a "Que pasaria si la idea fuera falsa?" La Fase 3 la transformo en "Como se construiria?" Y ahora, la Fase 3.5 la reduce a su forma mas esencial: "Leer primero la informacion correcta mejora realmente el resultado?" Esta es una pregunta que se puede responder con un experimento. Y eso es lo que la hace poderosa: es falsable.

La falsabilidad es el criterio que separa una hipotesis cientifica de una opinion. Una hipotesis que no puede ser refutada no es una hipotesis — es un articulo de fe. Nuestra hipotesis puede ser refutada: si en todos los escenarios probados, la estrategia MFT-first no ofrece ventaja sobre la secuencial, la hipotesis cae. No hay forma de salvarla con argumentos teoricos. Los datos diran la verdad. Y esa es exactamente la clase de hipotesis que queremos tener: una que, si es falsa, nos diga claramente que lo es, sin dejar espacio para interpretaciones autojustificativas.

Capítulo 2. H1: Hipótesis Central

H1: *En discos con presupuesto de lectura limitado por degradación física, una estrategia de adquisición basada en metadatos recuperables incrementa la cantidad de archivos recuperados y/o reduce el número de lecturas necesarias respecto a una estrategia secuencial.*

Cada palabra de esta formulación fue elegida deliberadamente. No es una hipótesis vaga sobre "mejor recuperación" — es una afirmación precisa sobre condiciones específicas, mecanismos específicos y métricas específicas. Vamos a descomponerla para entender exactamente qué afirma, qué no afirma, y bajo qué condiciones podría ser verdadera o falsa.

2.1 Descomposición de H1

Condición: "presupuesto de lectura limitado por degradación física"

Esta es la condición habilitante. H1 no afirma que la priorización por metadatos sea siempre mejor. Afirma que es mejor cuando existe una restricción real en la cantidad de datos que pueden leerse del disco antes de que este se deteriore irreversiblemente. Esta restricción puede provenir de varias fuentes: sectores que se vuelven ilegibles durante la lectura, un disco que se calienta y falla más rápido bajo carga sostenida, o un mecanismo de protección interna que limita los reintentos. En un disco sano, sin presupuesto limitado, ambas estrategias deberían producir resultados equivalentes — y eso es precisamente lo que esperamos encontrar. Si la priorización "ganara" incluso en discos sanos, sospecharíamos un sesgo en el benchmark.

Mecanismo: "metadatos recuperables"

La hipótesis no dice "metadatos" a secas — dice "metadatos recuperables". Esta distinción es crucial. Si el MFT está destruido, los metadatos no son recuperables, y el mecanismo que H1 propone no aplica. En ese escenario, la hipótesis no predice ventaja — y eso es correcto, no es una debilidad. Es una delimitación honesta del alcance. El mecanismo funciona cuando los metadatos están disponibles; cuando no lo están, el motor debe recurrir a estrategias alternativas como carving. Esta transición —de priorización basada en metadatos a carving cuando los metadatos fallan— es, de hecho, una de las decisiones de diseño más importantes que el motor debe tomar, y una de las áreas donde más valor puede aportar.

Resultado: "incrementa la cantidad de archivos recuperados y/o reduce el número de lecturas"

H1 afirma dos posibles resultados, no solo uno. El primero es cuantitativo: más archivos recuperados. El segundo es cualitativo: misma cantidad de archivos, pero con menos lecturas. El segundo resultado es tan importante como el primero. En un disco inestable, cada lectura es un riesgo: puede ser la última antes de que el disco falle completamente. Si Motor B recupera los mismos archivos que Motor A pero con un 40% menos de lecturas, eso ya es una ventaja enorme en la práctica clínica de recuperación de datos. Significa menos estrés sobre el disco, menos probabilidad de daño adicional, y más margen para reintentos si algo falla. La formulación "y/o" es deliberada: cualquiera de los dos resultados basta para validar H1.

2.2 Lo que H1 NO afirma

Es igualmente importante establecer lo que H1 no afirma, para evitar interpretaciones excesivamente amplias. H1 no afirma que la priorización MFT-first sea siempre superior a la secuencial. No afirma que funcione en todos los sistemas de archivos. No afirma que funcione cuando el MFT está destruido. No afirma que la ventaja sea grande — podría ser modesta, del orden del 3-5%, y aun así sería válida. H1 no es una promesa de producto; es una predicción sobre un mecanismo. Si la predicción es correcta, el mecanismo se puede incorporar a un producto. Si no, el mecanismo se descarta y buscamos otro.

H1 SI afirma	H1 NO afirma
Ventaja con presupuesto limitado	Ventaja en todos los escenarios
Ventaja cuando MFT es recuperable	Ventaja cuando MFT está destruido
Más archivos O menos lecturas	Ambas condiciones simultáneamente
Ventaja medible en NTFS	Ventaja en todos los filesystems
Mecanismo de priorización funciona	Mecanismo es siempre la mejor opción

2.3 Variables de H1

Para que H1 sea experimentalmente verificable, necesitamos definir sus variables operativas. La variable independiente es la estrategia de adquisición: secuencial (Motor A) vs. basada en metadatos (Motor B). La variable dependiente principal es la tasa de recuperación: porcentaje de archivos recuperados correctamente respecto al total de archivos en el disco. Las variables dependientes secundarias son el número total de lecturas realizadas, el tiempo hasta el primer archivo recuperado, el número de sectores leídos en vano, y el número de reintentos necesarios. Las variables de control incluyen el nivel de daño del disco, el sistema de archivos utilizado, el tipo de dispositivo (HDD vs SSD), y el presupuesto de lectura disponible.

Variable	Tipo	Descripción
Estrategia de adquisición	Independiente	Secuencial vs. MFT-first
Tasa de recuperación	Dependiente principal	% archivos recuperados correctamente
Número de lecturas	Dependiente secundaria	Total de operaciones de lectura
Tiempo al primer archivo	Dependiente secundaria	Segundos hasta primer archivo válido
Sectores leídos en vano	Dependiente secundaria	Lecturas que no contribuyen a recuperación
Reintentos	Dependiente secundaria	Intentos de lectura fallidos repetidos
Nivel de daño	Control	% de sectores dañados, % MFT eliminado
Filesystem	Control	NTFS, exFAT, APFS, EXT4
Tipo de dispositivo	Control	HDD SATA, SSD con TRIM

Variable	Tipo	Descripción
Presupuesto de lectura	Control	Máximo de sectores legibles antes de fallo

Capítulo 3. El simulador: resultados y límites

Antes de diseñar el siguiente experimento, es esencial entender que nos enseñó el simulador y, más importante aún, que no nos enseñó. El simulador fue el primer paso experimental: un modelo simplificado que nos permitió probar la hipótesis en condiciones controladas. Sus resultados fueron prometedores, pero sus limitaciones son significativas. Analizar ambas cosas con honestidad es la única forma de diseñar el siguiente experimento correctamente.

3.1 Resultados del simulador

El simulador modeló un disco NTFS con diferentes niveles de daño y presupuesto de lectura. Comparé dos motores: Motor A (lectura secuencial, sector por sector desde el inicio del disco) y Motor B (lectura priorizada: primero la MFT, luego los sectores de datos referenciados). Los resultados mostraron un patrón claro y coherente con la física del problema.

Escenario	Motor A (archivos)	Motor B (archivos)	Diferencia
Disco sano, presupuesto ilimitado	100%	100%	0% (empate)
Disco sano, presupuesto limitado	100%	100%	0% (empate)
20% sectores dañados, presupuesto bajo	67%	89%	+22%
40% sectores dañados, presupuesto bajo	34%	71%	+37%
60% sectores dañados, presupuesto bajo	12%	54%	+42%
Disco muriéndose, presupuesto crítico	0%	31%	+31%

Estos resultados son consistentes con la predicción de H1: la ventaja de Motor B aumenta a medida que el presupuesto de lectura se reduce y el daño al disco es mayor. En escenarios donde el presupuesto es generoso o el disco está sano, no hay diferencia — lo cual es lógico y, de hecho, tranquilizador. Si Motor B ganara siempre, sospecharíamos un sesgo en el benchmark. El patrón "empate cuando no hay restricción, ventaja cuando hay restricción" es exactamente lo que predice la física del problema.

3.2 El error detectado y corregido

Durante la ejecución del simulador, se detectó un error crítico: Motor A recuperaba 0 archivos en ciertos escenarios, no porque la estrategia secuencial fuera inherentemente incapaz de recuperar algo, sino porque el modelo no representaba correctamente la realidad. El motor secuencial no debía recuperar cero archivos — debía recuperar menos que Motor B, pero no cero. Este error fue detectado y corregido antes de sacar conclusiones, y ese proceso de corrección es posiblemente más valioso que los resultados mismos. Detectar que el modelo no representaba la realidad antes de publicar conclusiones es la diferencia entre ciencia y

autoengano.

La corrección del error no cambia la conclusión general — Motor B sigue siendo superior en escenarios con presupuesto limitado — pero cambia los números absolutos. Antes de la corrección, la diferencia parecía exagerada; después, la diferencia es más modesta pero igualmente significativa. Esto refuerza un principio fundamental: los números exactos importan menos que la dirección del efecto. Si la diferencia es del 3% o del 30%, la conclusión es la misma — pero solo si la medición es honesta.

3.3 Limitaciones del simulador

El simulador tiene limitaciones fundamentales que debemos reconocer antes de sobreinterpretar sus resultados. La primera y más importante: el simulador asume que Motor B "conoce perfectamente donde están los datos importantes". En un disco real, el MFT puede estar parcialmente corrupto, fragmentado, o ilegible en partes. El simulador no modela esta incertidumbre. La segunda limitación es que el simulador modela únicamente NTFS. No sabemos si los resultados se generalizan a otros filesystems como APFS, EXT4, o exFAT. La tercera limitación es que el simulador no modela la fragmentación de archivos ni la fragmentación del propio MFT. En un disco real, un archivo puede estar fragmentado en decenas de extensiones, y el MFT mismo puede estar fragmentado, lo que complica significativamente la tarea de Motor B.

Limitación	Impacto	Se resuelve en próximo experimento?
MFT perfectamente conocido	Sobrestima ventaja de Motor B	Si, con corrupción controlada
Solo NTFS	No sabemos si generaliza	Parcialmente, con otros FS
Sin fragmentación	Simplifica la realidad	Si, con imágenes reales
Sin journal corrupto	Ignora fuente de información	Si, con corrupción de journal
Sin SSD/TRIM	No aplica a SSD	No en esta fase

La pregunta clave que el simulador responde no es la misma que la pregunta original. La pregunta original era: "Leer primero la información correcta mejora la recuperación?" La pregunta que el simulador responde es: "Si conocemos perfectamente donde están los datos importantes, la priorización mejora la recuperación?" La diferencia entre ambas preguntas es la distancia entre la simulación y la realidad. Y esa distancia es precisamente lo que el siguiente experimento debe reducir.

Capítulo 4. El siguiente experimento: imágenes reales

El siguiente paso experimental no es tocar un disco físico. Es usar imágenes reales de disco. Las imágenes de disco ofrecen la reproducibilidad que un disco físico no puede ofrecer: podemos crear una imagen, corromperla de forma controlada, ejecutar ambos motores, y repetir el experimento exactamente con las mismas condiciones. Con un disco físico, cada lectura degrada el disco y hace el experimento irreproducible. Con imágenes, tenemos control total sobre las variables, y podemos iterar rápidamente.

4.1 Diseño del experimento

El experimento consiste en crear imágenes NTFS con contenido conocido, corromperlas de forma controlada, y ejecutar dos estrategias de adquisición sobre cada imagen. La comparación se realiza con checksums SHA-256 de cada archivo recuperado, verificando no solo que el archivo fue "recuperado" sino que fue recuperado correctamente. Un archivo con un solo byte incorrecto no cuenta como recuperado. Este criterio es más estricto que el del simulador, y deliberadamente así: queremos medir recuperación real, no aproximaciones.

Paso 1: Crear imagen NTFS base

Se crea una imagen de disco NTFS de 1 GB usando herramientas estándar (mkntfs). Se genera un conjunto de archivos de prueba con tipos y tamaños variados: documentos de texto, imágenes JPEG, archivos de oficina, binarios comprimidos. Cada archivo se registra con su checksum SHA-256, su ruta original, y su ubicación de sectores en el disco. Este registro es la "verdad fundamental" (ground truth) contra la cual se medirá la recuperación. Sin esta verdad fundamental, no podemos medir nada.

Paso 2: Corromper sectores específicos

Se aplican patrones de corrupción controlados a la imagen. Los patrones son reproducibles y cubren las dimensiones críticas que el simulador no modelaba: destrucción parcial del MFT (eliminando entradas específicas), fragmentación del MFT (forzando que el MFT se extienda en múltiples runs), corrupción del journal de NTFS (\$LogFile), sectores dañados en zonas de datos de usuario, y corrupción de entradas de directorio (INDEX). Cada patrón de corrupción se identifica con un código único para trazabilidad.

Paso 3: Ejecutar dos estrategias

Se implementan dos motores de lectura mínimos. Motor A lee la imagen secuencialmente desde el sector 0 hasta el final, con un presupuesto de lectura limitado. Motor B lee primero la MFT (o lo que quede de ella), identifica los sectores de datos referenciados, y los lee en orden de prioridad. Ambos motores se ejecutan sobre la misma imagen corrompida, con el mismo presupuesto de lectura. El presupuesto se simula limitando el número de sectores que cada motor puede leer antes de "detenerse".

Paso 4: Comparar resultados

La comparación se realiza en tres niveles. Primero, archivo-recuperado: un archivo se considera recuperado si su checksum SHA-256 coincide con el ground truth. Segundo, archivo-parcial: un archivo se considera parcialmente recuperado si se recuperaron más del 50% de sus sectores pero el checksum no coincide. Tercero, métricas de eficiencia: número de lecturas, sectores desperdiciados, tiempo hasta primer archivo. Este sistema de tres niveles evita la dicotomía simplista de "recuperado vs no recuperado" y captura matices que el simulador no podía.

4.2 Patrones de corrupción

Código	Patrón	Descripción	MFT afectado?
C01	MFT 20% eliminado	Entradas aleatorias del MFT borradas	Si
C02	MFT 40% eliminado	Entradas aleatorias del MFT borradas	Si

Código	Patrón	Descripción	MFT afectado?
C03	MFT 60% eliminado	Entradas aleatorias del MFT borradas	Si
C04	MFT fragmentado	MFT en múltiples runs no contiguos	Si (parcial)
C05	MFT zona ilegible	Parte del MFT en sectores dañados	Si (parcial)
C06	Journal corrupto	\$LogFile con datos inválidos	No
C07	Directorios destruidos	Entradas INDX corruptas	No
C08	Sectores datos dañados	Sectores de datos de usuario ilegibles	No
C09	Combinado: MFT+journal	C05 + C06 simultáneamente	Si
C10	Combinado: todo	C05 + C06 + C07 + C08	Si

Los patrones C01-C03 replican los escenarios del simulador pero con imágenes reales. Los patrones C04-C05 abren la dimensión de incertidumbre en el MFT que el simulador no modelaba. Los patrones C06-C07 evalúan fuentes de información alternativa (journal, INDX). Y los patrones C09-C10 combinan múltiples formas de daño para aproximarse a la complejidad de un disco real. Esta progresión de complejidad permite identificar exactamente donde la ventaja de Motor B comienza a degradarse.

4.3 Presupuestos de lectura

Cada imagen corrompida se prueba con múltiples presupuestos de lectura. El presupuesto se define como un porcentaje del total de sectores del disco. Un presupuesto del 100% significa que el motor puede leer todo el disco; un presupuesto del 20% significa que solo puede leer una quinta parte. Los presupuestos elegidos son: 100%, 80%, 60%, 40%, 20% y 10%. Esto permite trazar la curva de rendimiento de cada motor en función del presupuesto disponible, y observar cómo la ventaja de Motor B cambia con la restricción.

Capítulo 5. Métricas ampliadas

El simulador midió una única métrica: archivos recuperados. El siguiente experimento debe medir mucho más. La razón es doble: primero, porque la recuperación de datos no es solo una cuestión de cantidad, sino de eficiencia y seguridad. Segundo, porque la ventaja de Motor B puede manifestarse en dimensiones que no son capturadas por la tasa de recuperación bruta. Si Motor B recupera los mismos archivos que Motor A pero con un 40% menos de lecturas, eso ya es una ventaja enorme para discos inestables — y es una ventaja que el simulador no podía detectar.

5.1 Métricas primarias

Métrica	Unidad	Descripción
Tasa de recuperación	%	Archivos con checksum correcto / total archivos
Tasa de recuperación parcial	%	Archivos con >50% de sectores correctos / total

Metrica	Unidad	Descripción
Integridad de datos	Booleano	Checksum SHA-256 coincide con ground truth

La tasa de recuperación es la métrica principal, la que directamente valida o refuta H1. Pero la tasa de recuperación parcial es igualmente importante: un archivo parcialmente recuperado puede ser salvable con técnicas de reparación, mientras que un archivo no recuperado no tiene ninguna posibilidad. La integridad de datos es un filtro de calidad: un motor que "recupera" archivos con datos incorrectos no está recuperando, está corrompiendo. El checksum SHA-256 es el estándar de la industria para verificar integridad, y su uso es obligatorio en cada archivo recuperado.

5.2 Métricas de eficiencia

Metrica	Unidad	Descripción
Lecturas totales	Conteo	Número total de operaciones de lectura ejecutadas
Sectores desperdiciados	Conteo	Lecturas que no contribuyen a ningún archivo recuperado
Eficiencia de lectura	%	Sectores útiles / sectores totales leídos
Tiempo al primer archivo	Segundos	Tiempo desde inicio hasta primer archivo con checksum correcto
Tiempo total	Segundos	Tiempo desde inicio hasta agotar presupuesto

Las métricas de eficiencia capturan la dimensión que el simulador ignora: el costo de cada lectura. En un disco inestable, cada lectura es un recurso escaso. Si Motor A necesita 100,000 lecturas para recuperar 50 archivos, y Motor B necesita 60,000 lecturas para recuperar los mismos 50 archivos, Motor B es un 40% más eficiente — y eso significa que tiene 40,000 lecturas adicionales que podría usar para recuperar más archivos si el presupuesto lo permitiera. La eficiencia de lectura es la métrica que conecta directamente con la experiencia del usuario: un motor eficiente no solo recupera más, sino que cansa menos el disco y deja más margen para operaciones de rescate adicionales.

5.3 Métricas de estrés

Metrica	Unidad	Descripción
Reintentos	Conteo	Intentos de lectura fallidos repetidos sobre el mismo sector
Sectores con error	Conteo	Sectores que generaron error de lectura
Secuencia de errores	Lista	Patrón temporal de errores (para detectar degradación)
Lecturas consecutivas sin error	Conteo	Máximo de lecturas exitosas seguidas

Las métricas de estrés son relevantes para la futura integración con el motor de decisiones. Si el motor puede detectar que el disco está degradándose (más errores por unidad de tiempo, secuencia de errores creciente), puede ajustar su estrategia en tiempo real: priorizar los archivos más importantes, reducir los reintentos, o cambiar a un modo de adquisición más conservador. Estas métricas no validan H1 directamente, pero

preparan el terreno para las versiones futuras del motor de decisiones. El motor V1 (reglas expertas) usará estas métricas como entradas; el motor V2 (red bayesiana) las usará como evidencia; el motor V3 (ML) las usará como features de entrenamiento.

5.4 Escenario hipotético: la ventaja oculta

Imaginemos un escenario donde Motor A y Motor B recuperan la misma cantidad de archivos. A primera vista, H1 no tendría soporte. Pero si miramos las métricas de eficiencia: Motor A realizó 100,000 lecturas, Motor B realizó 60,000. Motor A desperdició 45,000 lecturas en sectores que no contenían datos útiles; Motor B desperdició solo 5,000. Motor B llegó al primer archivo en 12 segundos; Motor A tardó 180 segundos. En un disco inestable, esa diferencia de 40,000 lecturas es la diferencia entre un disco que sobrevive y un disco que muere durante la recuperación. Este escenario no es hipotético — es exactamente lo que esperamos encontrar en discos con presupuesto limitado.

Capítulo 6. Cuando el MFT falla

La mayor debilidad de Motor B es su dependencia del MFT. Si el MFT está destruido, Motor B no puede saber qué sectores priorizar, y su ventaja desaparece. Pero esta debilidad es, paradójicamente, una oportunidad. Porque la decisión de "cambiar a carving cuando el MFT falla" puede ser más valiosa que la priorización misma. Un motor que siempre intenta priorizar basándose en un MFT destruido no solo no obtiene ventaja — obtiene resultados peores que la lectura secuencial, porque pierde tiempo intentando leer metadatos ilegibles. Un motor que detecta que el MFT no es confiable y cambia automáticamente a carving está tomando una decisión inteligente que ningún motor actual toma de forma sistemática.

6.1 Árbol de decisiones de diagnóstico

El diagnóstico del estado del MFT es el primer paso de cualquier estrategia de recuperación inteligente. No se trata de "leer la MFT y priorizar" — se trata de "evaluar si la MFT es utilizable, y si no lo es, cambiar de estrategia". Este árbol de decisiones es, en esencia, el embrión del motor de decisiones V1: un conjunto de reglas expertas que determinan la estrategia de adquisición en función del estado del disco.

```
DIAGNOSTICO INICIAL
|
|-- MFT legible al 100%? --> PRIORIZAR MFT
| |
| |-- MFT fragmentada? --> Leer runs de MFT en orden
| |
| |-- MFT contigua? --> Leer MFT de una vez
|
|-- MFT parcialmente legible (40-99%)?
| |
| |-- Entradas MFT recuperables? --> PRIORIZAR MFT parcial
| | |
| | |-- Complementar con JOURNAL
| | |-- Complementar con INDX
| | |-- Complementar con BITMAP
```

```

| |
| |-- MFT mayormente ilegible? --> CARVING
| |
| |-- Usar journal para identificar archivos
| |-- Usar INDX para reconstruir directorios
|
|-- MFT destruido (<40%)?
|
|-- CARVING puro
| |
| |-- Buscar firmas de archivo (JPEG, PNG, PDF, DOCX...)
| |-- Reconstruir archivos por extensiones contiguas
|
|-- Complementar con fuentes secundarias
|
|-- $LogFile (journal) para operaciones recientes
|-- $Bitmap para sectores en uso
|-- $Secure para permisos
|-- INDX para estructura de directorios

```

Este árbol de decisiones no es un motor de IA. Es un conjunto de reglas expertas que codifican el conocimiento de la estructura NTFS. Cada decisión se basa en información observable: el motor puede determinar si el MFT es legible intentando leerlo, puede determinar si está fragmentado analizando sus runs, y puede determinar si el journal es utilizable verificando su integridad. No necesita aprender nada — necesita ejecutar un diagnóstico y actuar en consecuencia. Esta es la esencia del motor de decisiones V1: reglas que capturan el 90% del valor con el 10% del esfuerzo.

6.2 Fuentes de información alternativas

NTFS es un filesystem rico en metadatos. El MFT es la fuente primaria, pero no la única. Cuando el MFT falla, el motor tiene múltiples fuentes de información que puede consultar, cada una con diferente nivel de confiabilidad y utilidad. El journal (\$LogFile) contiene un registro de transacciones recientes, lo que permite identificar archivos que fueron modificados o creados poco antes del fallo. El bitmap de asignación (\$Bitmap) indica que clusters están en uso, lo que permite descartar sectores que no contienen datos. Las entradas INDX contienen índices de directorios, lo que permite reconstruir la estructura de carpetas incluso sin MFT. Y \$Secure contiene información de permisos, que puede ayudar a identificar la procedencia de los archivos.

Fuente	Archivo NTFS	Información que aporta	Disponibilidad típica
MFT	\$MFT	Ubicación de todos los archivos	80-95% en discos dañados
Journal	\$LogFile	Transacciones recientes	60-80% (se sobrescribe)
INDX	\$INDEX_ALLOCATION	Estructura de directorios	50-70% (fragmentado)
Bitmap	\$Bitmap	Clusters en uso	70-90% (simple, resiliente)
\$Secure	\$Secure	Permisos y atributos	40-60% (a menudo corrupto)

La jerarquia de fuentes es importante: el motor debe intentar primero la fuente mas confiable (MFT), y si no esta disponible, recurrir a las secundarias en orden de confiabilidad. Esta estrategia de "fallback en cascada" es la que diferencia un motor inteligente de uno que simplemente lee secuencialmente o que depende de una unica fuente de informacion. El motor de decisiones V1 no necesita IA para tomar esta decision — necesita una regla simple: "si el MFT es legible al X%, usalo; si no, pasa al journal; si no, pasa al INDX; si no, haz carving". El valor umbral de X% es un parametro que el experimento debe determinar.

6.3 El experimento de degradacion del MFT

Para medir el punto exacto donde Motor B pierde su ventaja, necesitamos un experimento especifico: degradar progresivamente el MFT y medir el rendimiento de ambos motores en cada nivel de degradacion. Los niveles de degradacion son: 0% (MFT intacto), 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, 100% (MFT completamente destruido). En cada nivel, medimos la tasa de recuperacion, la eficiencia de lectura, y el tiempo total. Este experimento nos permite trazar la "curva de degradacion" de Motor B y determinar el punto de inflexion donde su ventaja se convierte en desventaja.

La hipotesis es que existe un umbral critico — probablemente entre el 40% y el 60% de degradacion del MFT — donde Motor B debe cambiar a carving. Por debajo de ese umbral, Motor B es superior; por encima, Motor B pierde tiempo intentando leer un MFT que no aporta informacion util. Si este umbral existe y es medible, el motor de decisiones V1 puede usarlo como regla: "si el MFT esta degradado mas del X%, cambiar a carving". Esta regla simple, derivada de datos experimentales, captura gran parte del valor que un motor de IA capturaria con un modelo bayesiano, pero con una fraccion de la complejidad.

Capitulo 7. Matriz de ataque a H1

Una hipotesis robusta no es aquella que sobrevive a los escenarios favorables, sino la que sobrevive a los escenarios desfavorables. Esta seccion disena explicitamente escenarios para intentar destruir H1. Si H1 sobrevive a estos ataques, su credibilidad aumenta significativamente. Si H1 cae, habremos aprendido algo valioso: los limites exactos de la hipotesis. En ambos casos, el conocimiento avanza.

7.1 Escenarios de ataque

Ataque	Descripcion	Por que podria destruir H1
A1: MFT muy degradada	MFT destruida en 20+ runs no contiguos	Motor B pierde lecturas buscando runs de MFT
A2: MFT parcialmente degradado	Secciones del MFT en zonas danadas	Motor B no puede completar la MFT, informacion incompleta
A3: exFAT	Filesystem sin MFT, usa FAT	No hay MFT que priorizar, mecanismo de H1 no aplica
A4: APFS	Filesystem con estructura diferente	Estructura de metadatos diferente, estrategia no transferible
A5: EXT4	Filesystem con journal y superblock	Estrategia de priorizacion diferente, H1 no generaliza
A6: SSD con TRIM	SSD que borra bloques automaticamente	Datos no referenciados ya no existen fisicamente
A7: Journal con no hay	No hay fuente secundaria de informacion	Motor B no puede complementar MFT parcial

Ataque	Descripción	Por qué podría destruir H1
A8: Directorio corrupto	Entradas de directorio corruptas	No se puede reconstruir estructura de archivos
A9: Fragmentación extrema	Archivos en 50+ fragmentos	Motor B necesita muchas lecturas no contiguas
A10: MFT y datos mezclados	MFT y datos de usuario en mismas zonas	Priorizar MFT no aísla datos de usuario

Cada ataque apunta a una debilidad específica de H1. El ataque A1 (MFT fragmentada) prueba si la ventaja de priorización persiste cuando el mecanismo de priorización es más costoso. El ataque A3 (exFAT) prueba si H1 es específica de NTFS o si generaliza a otros filesystems. El ataque A6 (SSD con TRIM) prueba si la hipótesis siquiera tiene sentido en un contexto donde el hardware activamente destruye datos no referenciados. Y el ataque A10 (MFT y datos mezclados) prueba si la separación entre metadatos y datos que H1 asume es realista en práctica.

7.2 Priorización de ataques

No todos los ataques son igualmente urgentes. Los ataques A1 y A2 son prioritarios porque son extensiones directas del experimento con imágenes NTFS: no requieren un nuevo filesystem ni un nuevo tipo de dispositivo. Los ataques A3-A5 son prioritarios en un segundo nivel porque prueban la generalización de H1, pero requieren implementar parsers para cada filesystem. El ataque A6 (SSD con TRIM) es importante pero técnicamente complejo: simular el comportamiento de TRIM requiere un modelo de SSD que no tenemos todavía. Los ataques A7-A10 son variaciones del experimento NTFS que se pueden ejecutar en paralelo con los ataques principales.

Prioridad	Ataques	Requisito adicional	Esfuerzo estimado
Crítica	A1, A2	Ninguno (imágenes NTFS)	1-2 semanas
Alta	A3, A4, A5	Parsers de filesystem	3-4 semanas
Media	A7, A8, A9	Patrones de corrupción adicionales	1-2 semanas
Baja	A6, A10	Modelo de SSD, análisis de layout	4-6 semanas

7.3 Criterios de resultado por ataque

Para cada ataque, definimos explícitamente qué resultado refutaría H1, qué resultado la apoyaría, y qué resultado sería ambiguo. Esta definición previa es crucial: si no la hacemos antes de ejecutar el experimento, es tentador reinterpretar los resultados para que apoyen la hipótesis. Definir los criterios de antemano es la forma más efectiva de evitar el sesgo de confirmación.

Ataque	H1 refutada si	H1 apoyada si	Ambiguo si
A1	Motor B peor que A	Motor B mejor que A	Diferencia < 3%
A2	Motor B peor que A	Motor B mejor que A	Diferencia < 3%
A3	No hay MFT, H1 no aplica	Otro metadato funciona	Resultados mixtos

Ataque	H1 refutada si	H1 apoyada si	Ambiguo si
A4	Estrategia no transferible	Principio generaliza	Solo parcialmente
A5	Estrategia no transferible	Principio generaliza	Solo parcialmente
A6	TRIM destruye datos utiles	TRIM no afecta datos en uso	Depende del OS
A7	Motor B no puede recuperar	Motor B usa carving	Recuperación parcial
A8	Estructura irrecuperable	INDX parcial funciona	Reconstrucción parcial
A9	Costo de fragmentación > ventaja	Ventaja persiste	Ventaja solo en archivos grandes
A10	MFT y datos mezclados anulan ventaja	Separación suficiente	Ventaja reducida

Capítulo 8. Criterios de decisión

El propósito de todo este trabajo experimental es tomar una decisión. No una decisión teórica — una decisión práctica: construimos el motor de recuperación o no lo construimos? Y si lo construimos, con qué alcance? Esta sección define los criterios de decisión antes de ejecutar los experimentos, para que los resultados hablen por sí mismos.

8.1 Umbrales de decisión

Los umbrales se definen en términos de la métrica principal: diferencia en tasa de recuperación entre Motor B y Motor A, en escenarios con presupuesto de lectura limitado y MFT parcialmente recuperable. Los umbrales son deliberadamente conservadores: preferimos un falso negativo (descartar una idea que funcionaría) a un falso positivo (perseguir una idea que no funciona).

Resultado	Diferencia en tasa de recuperación	Decisión
H1 fuertemente apoyada	Motor B > Motor A por 10% o más	Construir motor completo, priorizar MFT-first
H1 moderadamente apoyada	Motor B > Motor A por 3-10%	Construir motor con MFT-first como opción, no único modo
H1 débilmente apoyada	Motor B > Motor A por 1-3%	Investigar más, no construir todavía
H1 no apoyada	Motor B = Motor A (diferencia < 1%)	Descartar MFT-first, explorar otras estrategias
H1 refutada	Motor B < Motor A	Descartar hipótesis, replantear enfoque

Estos umbrales no son arbitrarios. El 10% es el umbral donde la ventaja es inequívocamente significativa: en un disco con 10,000 archivos, 10% significa 1,000 archivos adicionales recuperados. El 3% es el umbral donde la ventaja es real pero modesta: 300 archivos adicionales, que puede no justificar la complejidad de un motor de priorización. Y el 1% es el umbral de ruido: cualquier diferencia menor al 1% puede deberse a variabilidad estadística, no a un efecto real. Estos umbrales se aplican a la tasa de recuperación; si la ventaja se manifiesta en métricas de eficiencia (menos lecturas), se aplican umbrales análogos: 40% menos lecturas es equivalente a 10% más archivos en términos de decisión.

8.2 Escenarios de decision

Escenario A: H1 sobrevive todos los ataques

Si Motor B es superior a Motor A en todos los escenarios de NTFS con presupuesto limitado y MFT parcialmente recuperable, y la ventaja persiste en los ataques A1-A2, la decision es clara: construir el motor de recuperacion con MFT-first como estrategia predeterminada. El motor de decisiones V1 se implementaria con reglas expertas basadas en los umbrales de degradacion del MFT identificados en el experimento. El roadmap avanzaria a la Fase 4 (construccion del motor) con alta confianza. Este es el mejor escenario, pero tambien el que requiere mayor escrutinio: si H1 sobrevive "demasiado bien", debemos verificar que no hay un sesgo sistematico en el diseno experimental.

Escenario B: H1 sobrevive parcialmente

Si Motor B es superior en algunos escenarios pero no en otros — por ejemplo, funciona bien con MFT degradado al 20-40% pero no al 60%+ — la decision es mas matizada. Construiriamos el motor, pero con un modo hibrido: MFT-first cuando el MFT es suficientemente recuperable, secuencial o carving cuando no lo es. El motor de decisiones V1 seria mas complejo pero mas realista: un arbol de decisiones con multiples ramas, cada una activada por un diagnostico del estado del disco. Este escenario es probablemente el mas realista, y no es una mala noticia: significa que H1 es verdadera en un dominio especifico, y que el motor debe ser inteligente sobre cuando aplicar la priorizacion.

Escenario C: H1 cae

Si Motor B no ofrece ventaja significativa en ningun escenario, o si la ventaja es consistentemente menor al 3%, la decision es descartar MFT-first como estrategia principal. Esto no significa que todo el trabajo fue en vano: los datos experimentales nos dicen algo valioso sobre la naturaleza del problema. Tal vez la priorizacion no funciona porque la estructura de NTFS no permite suficiente separacion entre metadatos y datos. Tal vez funciona pero solo con un presupuesto de lectura tan bajo que no es realista. En cualquier caso, el conocimiento avanza. Y la proxima hipotesis sera mejor porque esta informada por datos reales, no por intuicion.

8.3 La metrica de eficiencia como salvavidas

Hay un escenario que no esta en la tabla anterior: Motor A y Motor B recuperan la misma cantidad de archivos, pero Motor B lo hace con significativamente menos lecturas. En este escenario, H1 no es refutada — la formulacion "y/o" lo cubre explicitamente. Pero la decision practica es diferente: no construiriamos un motor que prioriza archivos, sino un motor que prioriza eficiencia. El producto seria diferente — no "recuperamos mas archivos" sino "recuperamos los mismos archivos con menos riesgo para el disco". Esta es una proposicion de valor valida, y posiblemente mas defendible en el mercado: los profesionales de recuperacion de datos valoran la seguridad del disco tanto como la tasa de recuperacion.

Este escenario es, de hecho, el que yo considero mas probable. La priorizacion MFT-first probablemente no recupere dramaticamente mas archivos — la diferencia en tasa de recuperacion puede ser modesta. Pero la diferencia en eficiencia puede ser enorme: leer solo los sectores que contienen datos utiles vs. leer todo el disco es la diferencia entre 60,000 y 100,000 lecturas. En un disco inestable, esa diferencia es la diferencia

entre un disco que sobrevive a la recuperacion y uno que no. Y si este escenario se confirma, la propuesta de valor del producto se redefine: no "recuperamos mas", sino "recuperamos mejor".

8.4 Roadmap post-experimento

Independientemente del resultado, el Benchmark Lab genera valor. Si H1 es apoyada, avanzamos a la Fase 4 con confianza. Si H1 es parcialmente apoyada, avanzamos con un alcance ajustado. Si H1 cae, replanteamos la hipotesis con datos reales. En ningun caso el trabajo es desperdiciado. El Benchmark Lab es, en este sentido, una inversion con retorno garantizado: si la hipotesis es correcta, nos da confianza; si es incorrecta, nos ahorra meses de trabajo en la direccion equivocada; y si es parcialmente correcta, nos dice exactamente donde enfocar el esfuerzo.

Resultado	Proxima fase	Alcance	Timeline estimado
H1 fuertemente apoyada	Fase 4: Construcción del motor	Motor completo con MFT-first	6-8 meses
H1 moderadamente apoyada	Fase 4: Motor híbrido	Motor con MFT-first + carving	8-10 meses
H1 debilmente apoyada	Más experimentos	Ampliar métricas y escenarios	2-3 meses adicionales
H1 no apoyada	Replanteamiento	Explorar otras estrategias	Indefinido
H1 refutada	Replanteamiento	Hipotesis nueva, experimento nuevo	Indefinido
Ventaja en eficiencia (no en costo)	Fase 4: Motor eficiente	Motor con priorización de lecturas	6-8 meses

Conclusion: Este documento no es un plan de producto. Es un protocolo experimental. La diferencia es fundamental: un plan de producto asume que la idea funciona; un protocolo experimental asume que no sabemos si funciona y lo verifica. Si H1 sobrevive a los ataques diseñados en la Matriz de Ataque, ya no será una intuición — será una hipótesis técnica con evidencia a favor. Y ese es el punto de partida real para construir un producto.